

Objektové programování

Co je objektové programování

- **objektové programování** = programovací přístup založený na objektech
- zkratka: **OOP**
- program se skládá z objektů, které spolu komunikují
- objekt spojuje:
 - data
 - chování
- používá se pro modelování reálných věcí

Příklad:

- v reálném světě máme auto
- auto má vlastnosti:
 - barva
 - rychlost
 - značka
- auto umí provádět akce:
 - jet
 - brzdit
 - troubit

Třída a objekt

Třída

- **třída** = šablona pro vytváření objektů
- definuje, jak bude objekt vypadat
- definuje, co bude objekt umět
- sama o sobě ještě není konkrétní objekt

Příklad:

```
class Auto
{
    public string Barva;
    public int Rychlost;

    public void Jed()
    {
        Console.WriteLine("Auto jede.");
    }
}
```

```
}  
}
```

Objekt

- **objekt** = konkrétní instance třídy
- vzniká podle šablony
- v C# se vytváří pomocí `new`

Příklad:

```
Auto mojeAuto = new Auto();  
  
mojeAuto.Barva = "červená";  
mojeAuto.Rychlost = 50;  
  
mojeAuto.Jed();
```

- `Auto` = třída
- `mojeAuto` = objekt / instance

Co obsahuje třída

Atributy / pole

- uchovávají data objektu
- popisují stav objektu

Příklad:

```
public string Jmeno;  
private int vek;
```

Metoda

- popisuje chování objektu
- jsou to funkce uvnitř třídy

Příklad:

```
public void Pozdrav()  
{  
    Console.WriteLine("Ahoj");  
}
```

Konstruktor

- speciální metoda pro vytvoření objektu
- volá se při použití `new`
- obvykle nastavuje počáteční hodnoty
- má stejný název jako třída
- nemá návratový typ
- konstruktory se nedědí

Příklad:

```
class Student
{
    public string Jmeno;

    public Student(string jmeno)
    {
        Jmeno = jmeno;
    }
}
```

Použití:

```
Student s = new Student("Jan");
```

Destruktor

- metoda volaná při zániku objektu
- v C# se běžně nepoužívá
- o uvolňování paměti se většinou stará **garbage collector**

Jmenné prostory

Co je namespace

- **jmenný prostor** = způsob organizace tříd a dalších typů
- v C# se zapisuje jako `namespace`
- pomáhá zabránit konfliktům názvů
- umožňuje rozdělit kód do logických celků

Příklad:

```
namespace Skola.Modely
{
    public class Student
```

```
{  
    public string Jmeno;  
}  
}
```

Použití namespace

- pokud chceme použít třídu z jiného jmenného prostoru, použijeme `using`

Příklad:

```
using System;  
using Skola.Modely;
```

Příklad z .NET:

```
using System.Collections.Generic;
```

- `System` obsahuje základní třídy
- `System.Collections.Generic` obsahuje například `List<T>`

Zapouzdření

Co je zapouzdření

- **zapouzdření** = skrytí vnitřního stavu objektu
- anglicky **encapsulation**
- objekt neodhaluje vše přímo
- ven nabízí jen bezpečné rozhraní
- chrání data před nechtěnou změnou

Příklad:

```
class Ucet  
{  
    private decimal zustatek;  
  
    public void Vloz(decimal castka)  
    {  
        if (castka > 0)  
        {  
            zustatek += castka;  
        }  
    }  
  
    public decimal ZiskejZustatek()  
}
```

```
{  
    return zustatek;  
}  
}
```

- `zustatek` je skrytý
- měnit ho lze jen přes metody
- objekt si hlídá pravidla

Modifikátor přístupu

Modifikátor	Význam
<code>private</code>	přístup pouze uvnitř stejné třídy
<code>protected</code>	přístup uvnitř třídy a jejích potomků
<code>public</code>	přístup odkudkoliv
<code>internal</code>	přístup v rámci stejného projektu / assembly

Vlastnosti

Co jsou vlastnosti

- **vlastnost** = property
- v C# bezpečný způsob přístupu k datům objektu
- navenek vypadá jako proměnná
- uvnitř může obsahovat logiku
- používá `get` a `set`

Příklad:

```
class Student  
{  
    private int vek;  
  
    public int Vek  
    {  
        get { return vek; }  
        set  
        {  
            if (value >= 0)  
            {  
                vek = value;  
            }  
        }  
    }  
}
```

```
}  
}
```

Použití:

```
Student s = new Student();  
s.Vek = 18;  
Console.WriteLine(s.Vek);
```

Automatické vlastnosti

- zkrácený zápis property
- C# si vnitřní pole vytvoří sám

Příklad:

```
class Student  
{  
    public string Jmeno { get; set; }  
    public int Vek { get; set; }  
}
```

Read-only vlastnost

- vlastnost může mít veřejný `get`
- a soukromý `set`
- zvenku jde hodnotu číst
- ale nejde ji přímo měnit

Příklad:

```
public string Jmeno { get; private set; }
```

Dědičnost

Co je dědičnost

- **dědičnost** = možnost vytvořit novou třídu na základě existující třídy
- potomek přebírá členy předka
- potomek může přidat vlastní členy
- potomek může změnit chování některých metod

Příklad:

```

class Clovek
{
    public string Jmeno;

    public void Pozdrav()
    {
        Console.WriteLine("Ahoj, jsem " + Jmeno);
    }
}

class Student : Clovek
{
    public string Trida;
}

```

Použití:

```

Student s = new Student();
s.Jmeno = "Jan";
s.Trida = "4.A";
s.Pozdrav();

```

Rodič a potomek

- rodičovská třída:
 - předek
 - base class
 - superclass
- odvozená třída:
 - potomek
 - derived class
 - subclass

Dědičnost v Csharp (C#)

- C# umožňuje dědit pouze z jedné třídy
- tomu se říká jednoduchá dědičnost
- třída ale může implementovat více rozhraní
- konstruktor předka se volá dříve než konstruktor potomka

Volání konstruktoru předka

```

class Clovek
{
    public string Jmeno;
}

```

```

    public Clovek(string jmeno)
    {
        Jmeno = jmeno;
    }
}

class Student : Clovek
{
    public string Trida;

    public Student(string jmeno, string trida) : base(jmeno)
    {
        Trida = trida;
    }
}

```

Polymorfismus

Co je polymorfismus

- **polymorfismus** = mnohotvarost
- stejný kód může pracovat s různými typy objektů
- objekty mohou na stejnou zprávu reagovat různě
- často souvisí s dědičností a rozhraními

Příklad:

```

class Zvire
{
    public virtual void VydatZvuk()
    {
        Console.WriteLine("Zvuk zvířete");
    }
}

class Pes : Zvire
{
    public override void VydatZvuk()
    {
        Console.WriteLine("Haf");
    }
}

class Kocka : Zvire
{
    public override void VydatZvuk()
    {

```



```
        Console.WriteLine("Mňau");  
    }  
}
```

Použití:

```
List<Zvire> zvirata = new List<Zvire>  
{  
    new Pes(),  
    new Kocka()  
};  
  
foreach (Zvire zvire in zvirata)  
{  
    zvire.VydatZvuk();  
}
```

Výstup:

```
Haf  
Mňau
```

virtual a override

- **virtual**
 - metoda v předkovi
 - dovoluje přepsání v potomkovi
- **override**
 - metoda v potomkovi
 - mění chování metody z předka

Přetížení

Co je přetížení

- **přetížení** = více metod se stejným názvem
- metody se liší:
 - počtem parametrů
 - typem parametrů
 - pořadím parametrů
- návratový typ sám o sobě nestačí pro přetížení

Příklad:

```

void Vypis(int cislo)
{
    Console.WriteLine(cislo);
}

void Vypis(string text)
{
    Console.WriteLine(text);
}

void Vypis(string text, int pocet)
{
    for (int i = 0; i < pocet; i++)
    {
        Console.WriteLine(text);
    }
}

```

Použití:

```

Vypis(10);
Vypis("Ahoj");
Vypis("Ahoj", 3);

```

Abstrakce

Co je abstrakce

- **abstrakce** = zjednodušení reality
- zaměřuje se na podstatné vlastnosti a chování
- skrývá zbytečné detaily
- v OOP se realizuje například pomocí:
 - abstraktních tříd
 - rozhraní
 - zapouzdření

Příklad:

- při práci s autem řešíme:
 - Nastartuj()
 - Jed()
 - Zastav()
- neřešíme všechny vnitřní detaily motoru

Abstraktní třída

Co je abstraktní třída

- **abstraktní třída** = neúplná třída
- používá se jako společný základ pro potomky
- nelze z ní vytvořit instanci
- může obsahovat:
 - hotové metody
 - abstraktní metody
 - atributy
 - vlastnosti
 - konstruktor

Příklad:

```
abstract class Zvire
{
    public string Jmeno { get; set; }

    public void Spi()
    {
        Console.WriteLine("Zvíře spí.");
    }

    public abstract void VydatZvuk();
}
```

Abstraktní metoda

- nemá tělo
- obsahuje jen předpis
- potomek ji musí implementovat pomocí `override`

Příklad:

```
class Pes : Zvire
{
    public override void VydatZvuk()
    {
        Console.WriteLine("Haf");
    }
}
```

Interface

- **interface** = rozhraní

- definuje smlouvu
- říká, co musí třída umět
- neřeší konkrétní implementaci
- třída musí implementovat členy interface
- třída může implementovat více interface

Příklad:

```
interface IZvire
{
    void VydatZvuk();
}
```

Implementace:

```
class Pes : IZvire
{
    public void VydatZvuk()
    {
        Console.WriteLine("Haf");
    }
}
```

Kdy použít interface

- když chceme předepsat společné chování
- když nechceme řešit společného předka
- když chceme umožnit více různých implementací
- když potřebujeme, aby různé třídy šly používat stejným způsobem

Příklady:

- IComparable
- IEnumerable
- IDisposable

Abstraktní třída vs. interface

Vlastnost	Abstraktní třída	Interface
Instance	nelze vytvořit	nelze vytvořit
Hotový kód	může obsahovat	obvykle neřeší logiku
Atributy	může mít	běžně ne
Dědičnost	jen jedna třída	více interface

Vlastnost	Abstraktní třída	Interface
Použití	společný základ	smlouva / schopnost

Statika

Co je static

- **static** znamená, že člen patří třídě
- nepatří konkrétní instanci objektu
- není potřeba vytvářet objekt pomocí `new`
- používá se pro společné metody nebo hodnoty

Příklad:

```
Math.Sqrt(16);  
Console.WriteLine("Ahoj");
```

Statická metoda

```
class Matematika  
{  
    public static int Secti(int a, int b)  
    {  
        return a + b;  
    }  
}
```

Použití:

```
int vysledek = Matematika.Secti(3, 5);
```

Statická vlastnost / proměnná

- sdílí se mezi všemi instancemi třídy
- existuje jen jednou pro celou třídu

Příklad:

```
class Student  
{  
    public static int PocetStudentu = 0;  
  
    public Student()  
    {
```

```
        PocetStudentu++;  
    }  
}
```

Sealed

Co je sealed

- `sealed` znamená, že z třídy nelze dědit
- používá se, když nechceme dovolit vytvoření potomků

Příklad:

```
sealed class Nastaveni  
{  
}
```

Generika

Co je generika

- **generika** umožňuje psát kód nezávislý na konkrétním typu
- konkrétní typ se doplní až při použití
- zapisuje se pomocí `<T>`
- zvyšuje znovupoužitelnost kódu
- zachovává typovou bezpečnost

Příklad:

```
List<int> cisla = new List<int>();  
List<string> jmena = new List<string>();
```

- `List<T>` je obecná šablona
- `List<int>` je list pro celá čísla
- `List<string>` je list pro texty

Generická metoda

```
T VratPrvni<T>(T[] pole)  
{  
    return pole[0];  
}
```

Použití:

```
int[] cisla = { 1, 2, 3 };
string[] jmena = { "Jan", "Eva" };

Console.WriteLine(VratPrvni(cisla));
Console.WriteLine(VratPrvni(jmena));
```

Praktické ukázky

1. Třída a objekt

```
class Auto
{
    public string Barva;
    public int Rychlost;

    public void Jed()
    {
        Console.WriteLine("Auto jede.");
    }
}

Auto auto = new Auto();
auto.Barva = "červená";
auto.Rychlost = 50;
auto.Jed();
```

2. Zapouzdření přes property

```
class Student
{
    private int vek;

    public int Vek
    {
        get { return vek; }
        set
        {
            if (value >= 0)
            {
                vek = value;
            }
        }
    }
}
```

3. Dědičnost a polymorfismus

```

class Zvire
{
    public virtual void VydatZvuk()
    {
        Console.WriteLine("Zvuk");
    }
}

class Pes : Zvire
{
    public override void VydatZvuk()
    {
        Console.WriteLine("Haf");
    }
}

Zvire zvire = new Pes();
zvire.VydatZvuk();

```

4. Interface

```

interface IPracovnik
{
    void Pracuj();
}

class Programator : IPracovnik
{
    public void Pracuj()
    {
        Console.WriteLine("Programuji.");
    }
}

```

5. Generická metoda

```

T VratPrvni<T>(T[] pole)
{
    return pole[0];
}

```